ARTEFACT / AI ENGINEER TECHNICAL CASE

Empório da Música
AI Customer Support Agent

Fernando d'Ávila Filho
AI Engineer - Full-Stack Generative AI Applications

Public repository: https://github.com/TaldoFernando/artefact

Recorded runtime: **5dc7d6b6bd0e**. Source code, tests and exact conversations are available in the repository.

Problem and solution

Mel helps the fictitious Campo Grande music store answer recurring questions about instruments, prices, availability, order status and store rules. The prototype is a Python command-line conversation using local Ollama/Qwen3 8B, six read-only tools and evidence from supplied CSVs and the original policy PDF.

Architecture and implemented capabilities

Component	Role
CLI and context	One local session; four complete turns; reset clears history and recommendation context.
Ollama / Qwen3 8B	Interprets messages and calls allowlisted tools; no paid inference API or API key.
Six read-only tools	search_products, get_product_details, get_active_promotions, recommend_products, order_status, search_policies.
Policy retrieval	Lexical PDF pages plus reviewed English notes enabled only for the exact source PDF hash.
Answer composition	Exact prices, stock and order facts; separate policy guidance; mixed questions preserve all verified components.
Controls	Validated arguments, ownership checks, provider-error handling and bounded tool rounds.

Why this design

Native function calling provides explicit query schemas. Python handles Decimal pricing, active-promotion flags, nonstacking PIX discounts and ownership. Lexical retrieval keeps the eight-page manual searchable without an embedding service. Reviewed English notes retain policy conditions and source-page references; their hash guard prevents reuse with a changed PDF.

Technology and execution

Python 3.11+, standard-library HTTP/CSV/JSON/Decimal tools, pypdf, Ollama and Qwen3 8B. Tests use unittest and unittest.mock. ReportLab builds this document. The model download is about 5.2 GB; runtime memory depends on hardware and context. Evaluation used a macOS host with 24 GiB RAM; no universal minimum is claimed.

From the repository root, with Python 3.11+ and Ollama installed:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
ollama serve # separate terminal, if not already running
ollama pull qwen3:8b
python scripts/validate_data.py
python -m emporio.cli
```

Configuration: OLLAMA_HOST=http://localhost:11434 and OLLAMA_MODEL=qwen3:8b. The app reads process environment and does not automatically load .env. Chat requires a running Ollama; /reset and /quit are CLI commands. Full setup and troubleshooting are in the README.

Evidence, business rules and privacy

The challenge supplies a fictional music-store dataset: six CSVs and a policy PDF, preserved byte-for-byte. Customer names, emails and phones are neither authentication credentials nor tool output. The repository provenance note records the candidate statement without treating it as independent certification or a publication license.

Only active, stocked instruments are presented as available. Promotional prices use explicit active flags; a PIX discount never stacks with an active campaign. Follow-up recommendations remain within the original budget when source prices change. The source files are historical snapshots, not live inventory or carrier data.

Order details require identity supplied by trusted application code and an ownership check. The CLI has no login. The order example explicitly uses a test identity fixture, not real authentication. Estimated delivery is never treated as actual receipt.

The manual conflicts on contacts, installment minimums and exchange/warranty handling, and leaves local freight at exactly BRL 500 undefined. The assistant preserves uncertainty and requests team confirmation. Customized online returns require review; the prototype cannot approve, process or register a return.

AI assistance

OpenAI Codex assisted with source inspection, implementation, prompts, regression tests, real-model evaluation and English documentation. The candidate chose local Ollama without a paid API and is responsible for reviewing the code, results and design decisions before submission. Candidate review is not assumed.

Executed validation

Offline suite: 55 executed, 55 passed, 0 failed. Recorded on Python 3.12.14 / macOS. Dependency consistency, data validation, compilation and CLI start/reset/exit were also checked; logs are linked from the repository reports.

Unit tests include unavailable products, active/inactive offers, PIX, budgets, owned/missing/foreign orders, mixed sources, follow-ups, malformed model output, connectivity failures, empty evidence and changed-PDF handling. Scripted model fixtures test the protocol; real model conversations are evaluated separately.

Actual local-model scenario	Reviewed result
Filter acoustic guitars by price and availability, recommend from prior candidates, and combine catalog and payment policy.	PASS
Retrieve address, Saturday hours, accepted payment methods and PIX conditions across a conversation.	PASS
Require authentication, then demonstrate owned, nonexistent and foreign orders using an explicitly trusted test identity.	PASS
Apply online regret-return rules, address customized instruments without denying rights, and acknowledge the absence of transaction tools.	PASS
Reject unrelated merchandise/weather and data exfiltration, then resume an electric-guitar search for a beginner.	PASS

The five examples contain 20 exact application turns, with model digest, runtime commit, tool traces, source comparison and limitations. Failed earlier runs remain in the repository. Runtime revision: **5dc7d6b6bd0e**.

```
python -m unittest discover -s tests -v
python scripts/evaluate.py
python scripts/verify_run.py PATH_TO_RUN_JSON
```

Relevant limitations

There is no production authentication, WhatsApp integration, live ERP/carrier connection, refund processor or operational handoff. Lexical topic matching may miss paraphrases; unknown policy prose can be imprecise. Reviewed notes are disabled when the PDF changes and must then be re-reviewed. Four turns do not guarantee token-budget fit. Dependency ranges and model tags are not a full reproducibility lock.

Beginner recommendations use explicit source descriptions. If no electric-guitar description establishes beginner suitability, Mel shows the real options and asks for a budget instead of inventing advice. Read-only tools and prompts reduce risk but do not prove universal prompt-injection resistance.

Evidence in the repository

The README contains setup and technical decisions. The examples index links all five transcripts and reproduction commands; reports retain earlier failures, corrections and command logs. Source-based scenario review is separate from automated record/hash verification.